

EECS 4101/5101

Prof. Andy Mirzaian



**Computer Science
and Engineering**

120 Campus Walk

Nearest Neighbors & Closest Pair

References:

- Lecture Note 8 [[LN8](#)]
- [CLRS] chapter 33

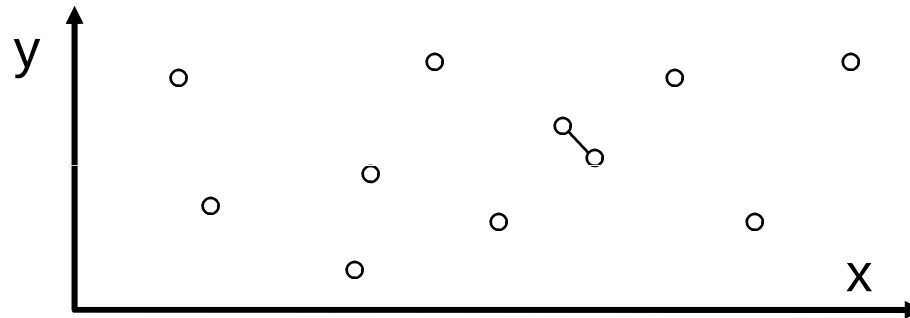
Applications:

- ☐ Proximity clustering
- ☐ Pattern Recognition
- ☐ Related to:
 - Euclidean Minimum Spanning Tree,
 - Relative Neighborhood Graph,
 - Delaunay Triangulation

The Closest Pair of Points

Input: A set $P = \{p_1, p_2, \dots, p_n\}$ of n points in the plane.
Each point is given by its x & y coordinates $p_i = (x_i, y_i)$, $i=1..n$.

Output: The closest pair of points in P , i.e., the pair (p_i, p_j) in P , $i \neq j$,
with minimum Euclidean distance $d(p_i, p_j) = ((x_i - x_j)^2 + (y_i - y_j)^2)^{1/2}$.



In 1 dimension: The Min-Gap Problem.

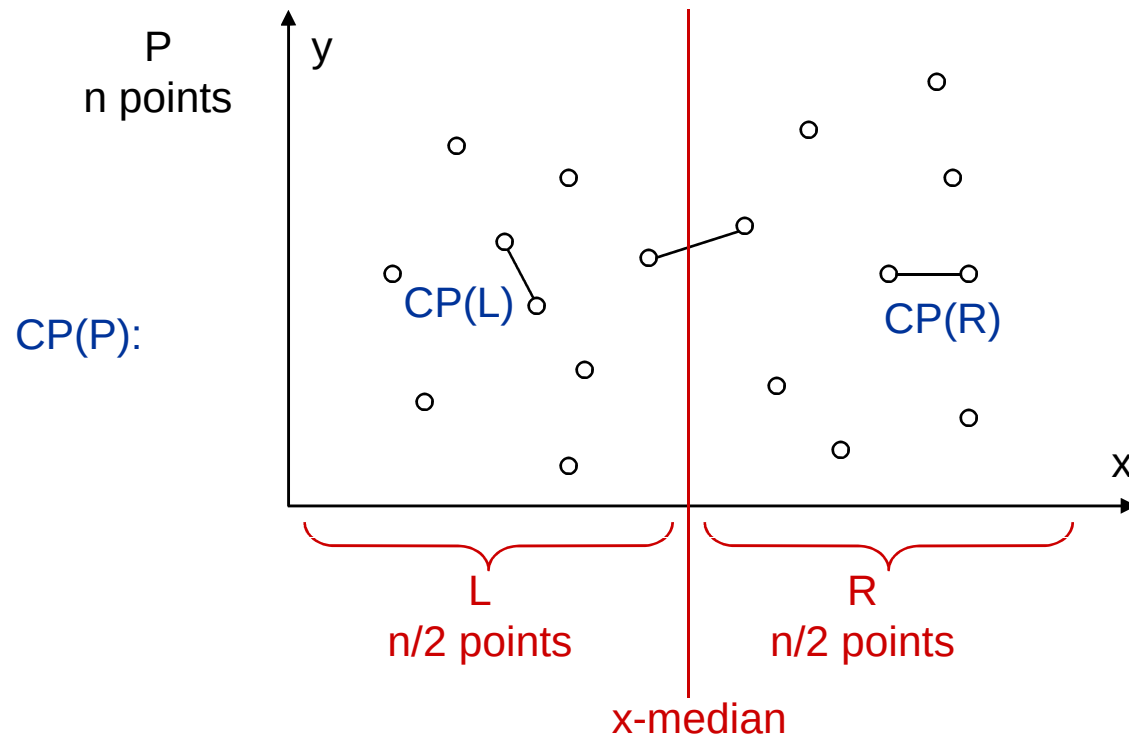
Known lower bound on time complexity: $\Omega(n \log n)$. Discussed later.
Matching upper bound (by sorting first): $O(n \log n)$.

In 2 dimensions it is at least as hard.

Brute Force: Try every pair. That takes $\Theta(n^2)$ time.

Divide-&-Conquer P.T.O.

CP: Divide-&-Conquer



The closest pair of points are either:

- a) both in the left half L, or [recursive call on L]
- b) both in the right half R, or [recursive call on R]
- c) one is in L and the other in R. [Combine L & R]

Divide-&-Conquer template

Algorithm **ClosestPair(P)**

Pre-Condition: P is a finite set of points in the plane

Post-Condition: Output is the closest pair of points in P

1. Pre-Sort points in P on their x-coordinates (lexicographically next on y)
 2. **return** CP(P)
- end**

Procedure **CP(P)**

Pre-Condition: P is a x-sorted finite set of points

Post-Condition: Output is the closest pair of points in P

- 3.Base: **if** $|P| < 10$ **then return** answer by brute-force in $\Theta(1)$ time
 - 4.Divide: Partition P at its x-median value into sets L and R, $|L| \approx |R| \approx |P|/2$
 - 5.Conquer: $SolL \leftarrow CP(L)$; $SolR \leftarrow CP(R)$
 - 6.Combine: $Sol \leftarrow \text{MERGE}(SolL, SolR)$
 - 7.Output: **return** Sol
- end**
-

$$T(n) = 2T(n/2) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n) \text{ time.}$$

Divide-&-Conquer template

Algorithm

Pre-Cond

Post-Cond

1. Pre

2. ret

end

Post-Cond of MERGE must imply Post-Cond of CP(P).

On the other hand,

Post-Cond's of CP(L) and CP(R) must imply
Pre-Cond of MERGE.

Strengthen Post-Cond of CP (**CP(L) & CP(R)**)
to help reduce the burden on MERGE!

Procedure CP(P)

Pre-Condition: P is a finite set of points

Post-Condition: Output is the closest pair of points in P

3.Base: if $|P| < 1$ then return answer by brute-force in $\Theta(1)$ time

4.Divide: Partition P at its x-median value into sets L and R, $|L| \approx |R| \approx |P|/2$

5.Conquer: SolL $\leftarrow$ CP(L); SolR $\leftarrow$ CP(R)

6.Combine: Sol $\leftarrow$ **MERGE(SolL, SolR)**

7.Output: return Sol

end

Our aim

$$T(n) = 2T(n/2) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n) \text{ time.}$$

Strengthen CP Post-Condition

Procedure **CP(P)**

Pre-Condition: P is a x -sorted finite set of points

Post-Condition: Output is the closest pair of points in P , and
 P is rearranged into y -sorted order.

3.Base: **if** $|P| < 10$ **then return** answer by brute-force in $\Theta(1)$ time

4.Divide: Partition P at its x -median value into sets L and R , $|L| \approx |R| \approx |P|/2$
 § Now L & R are x -sorted.

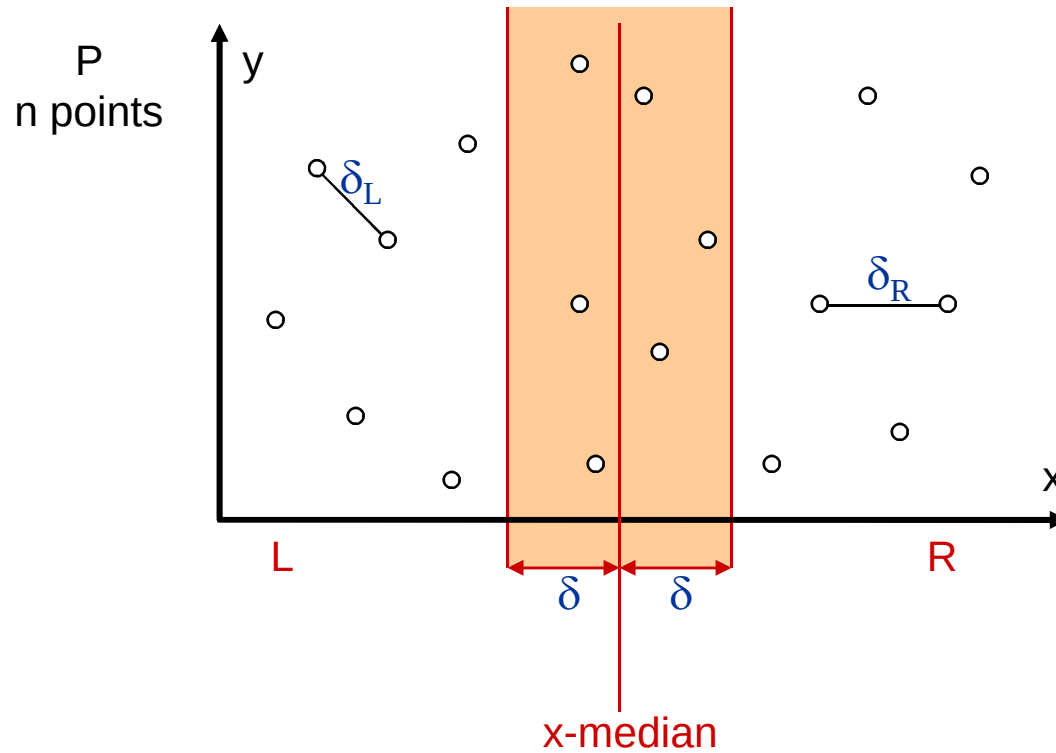
5.Conquer: $SolL \leftarrow \mathbf{CP}(L)$; $SolR \leftarrow \mathbf{CP}(R)$
 § Now L & R are y -sorted.
 § MERGE can y -merge L & R , and ...

6.Combine: $Sol \leftarrow \mathbf{MERGE}(SolL, SolR)$
 § Now $P = L \cup R$ is y -sorted, and ...

7.Output: **return** Sol

end

MERGE



MERGE(L, R):

$$\delta = \min \{ \delta_L, \delta_R \}$$

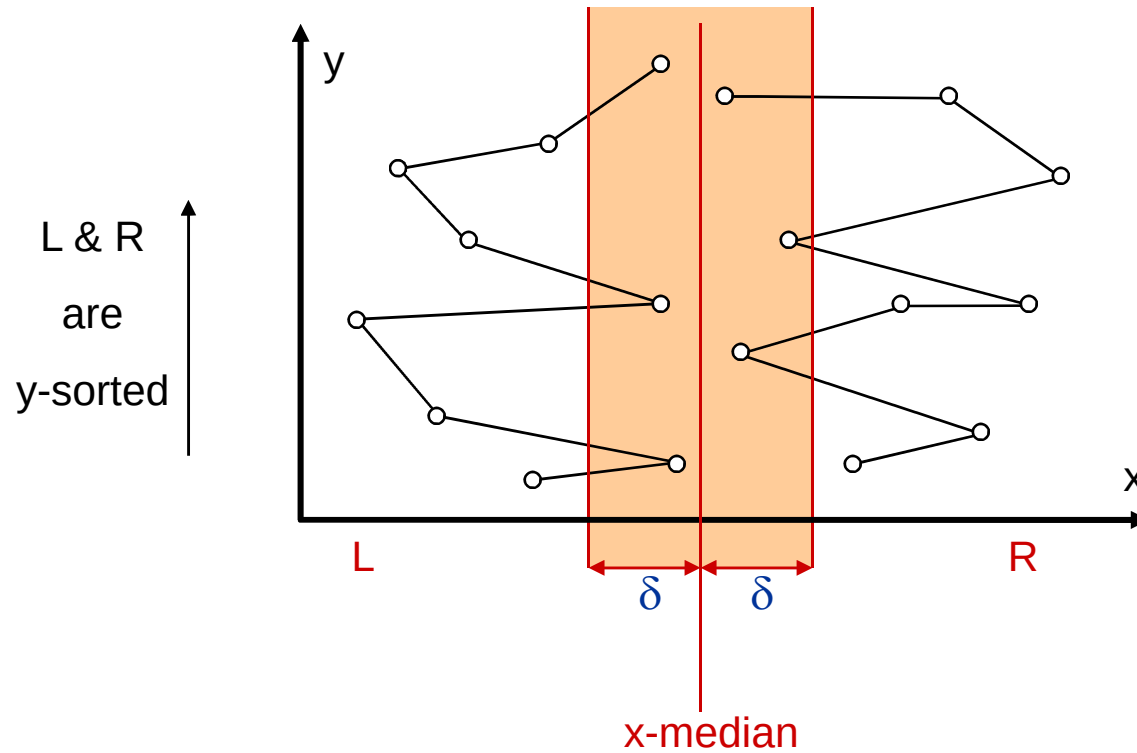
...

...

end

Can we do it
in $O(n)$ time?

MERGE



MERGE(L, R):

$$\delta = \min \{ \delta_L, \delta_R \}$$

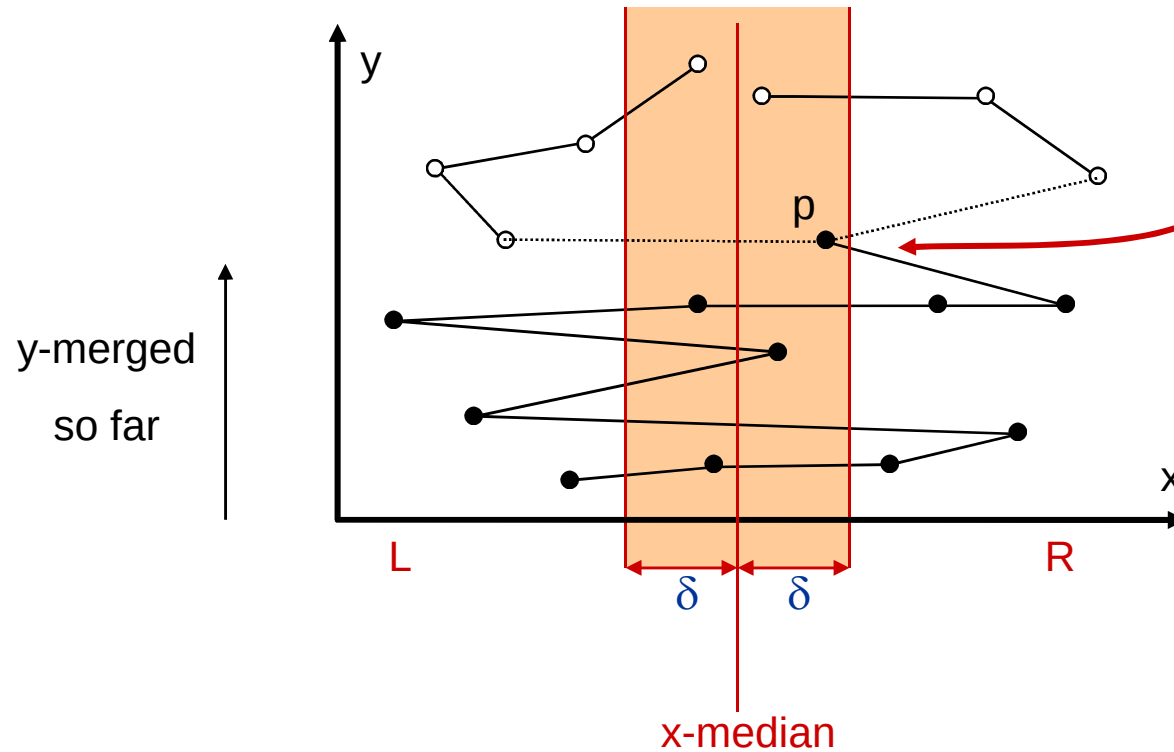
...

...

end

Can we do it
in $O(n)$ time?

MERGE



p is the latest point merged so far.

If p is in the 2δ vertical slab:

is p too close to a merged point on the opposite side?

MERGE can't afford checking p against every merged point!

Is there a short-cut?

MERGE(L, R):

$\delta = \min \{\delta_L, \delta_R\}$
y-merge L & R

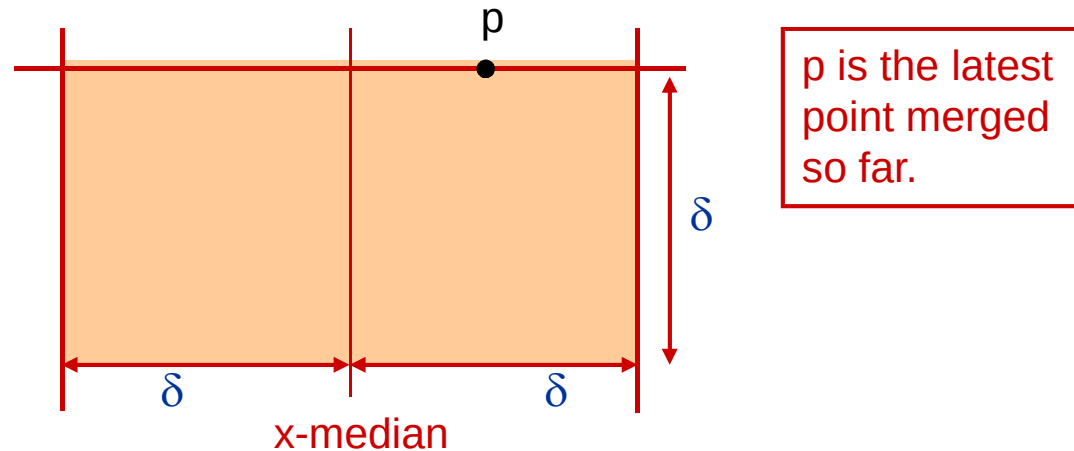
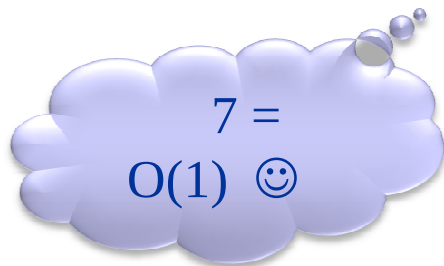
...

end

Can we do it in $O(n)$ time?

MERGE

FACT: There can be at most 7 points (excluding p) in the shaded 2δ -by- δ rectangle shown below. **Why?**



MERGE:

- Maintain the (up to) 7 latest merged points that fall within the 2δ vertical slab.
 - **If** next point p being merged falls within this slab
then compare p against the “7 points”; update closest pair;
add p to the “7 point” list (remove the now lowest 8th from the list).
 - Add p to the merged list and move up to the next point.
 - Each point is y -merged in $O(1)$ time.
MERGE takes $O(n)$ time.
- Therefore, CP takes $O(n \log n)$ time.

All Nearest Neighbors Problem (ANNP)

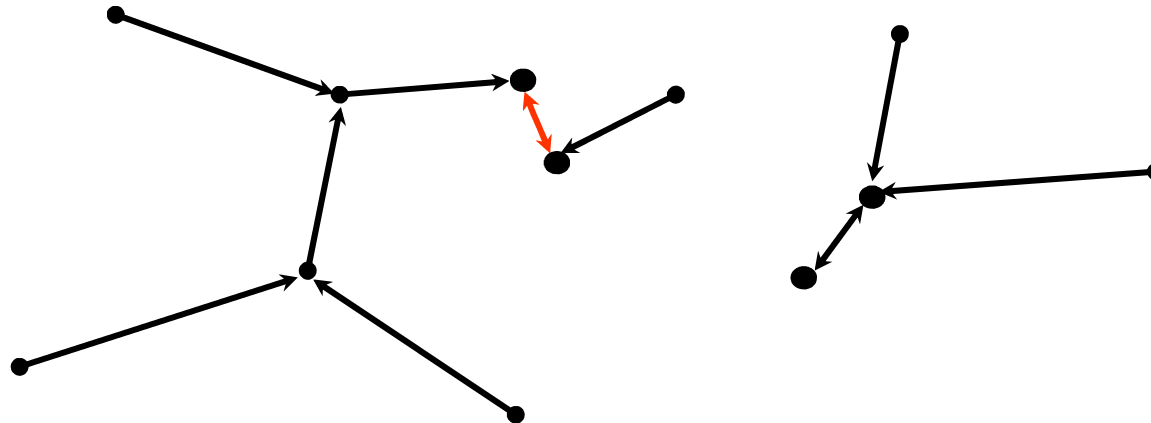
Input: A set $P = \{ p_1, p_2, \dots, p_n \}$ of n points in the plane, $p_i = (x_i, y_i)$, $i=1..n$.

Output: Nearest Neighbor $NN(p_i)$ of p_i , for all $i=1..n$.

$NN(p_i) = p_j$, for some $p_j \in P - \{p_i\}$, s.t. $d(p_i, p_j) \leq d(p_i, p_k) \forall p_k \in P - \{p_i\}$.
with Euclidean distance: $d(p_i, p_j) = ((x_i - x_j)^2 + (y_i - y_j)^2)^{1/2}$.

Related Problem: Closest Pair:

Find the closest pair (p_i, p_j) : one that minimizes $d(p_i, p_j)$, $i \neq j$.



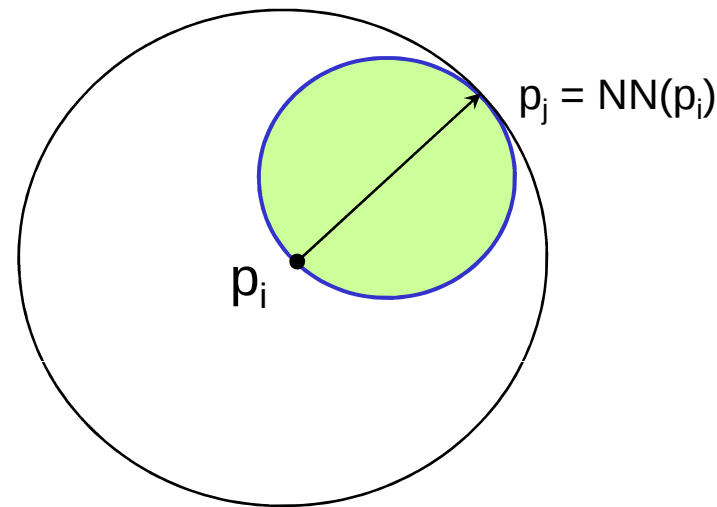
An All Nearest Neighbors Graph & **Closest Pair**.

All Nearest Neighbors Problem (ANNP)

- ❑ All Nearest Neighbors Graph (ANNG) can be viewed as a directed sub-graph of the complete graph. The latter has $O(n^2)$ edges with Euclidean edge lengths.
- ❑ Once we have ANNG, the closest pair (CP) can be obtained in $O(n)$ additional time (since CP is the shortest edge among the n edges of ANNG).
- ❑ [CLRS] describes an $O(n \log n)$ time divide-&-conquer algorithm for CP, due to [Shamos-Hoey 1975]. See pages 3-10 of this Slide.
- ❑ We will describe an $O(n \log n)$ time algorithm for ANNP by the lifting method.

The empty circle property

□ If $p_j = \text{NN}(p_i)$, then the circle with diameter (p_i, p_j) is an empty circle (no point of P is in the interior of that circle and only p_i, p_j are on it).

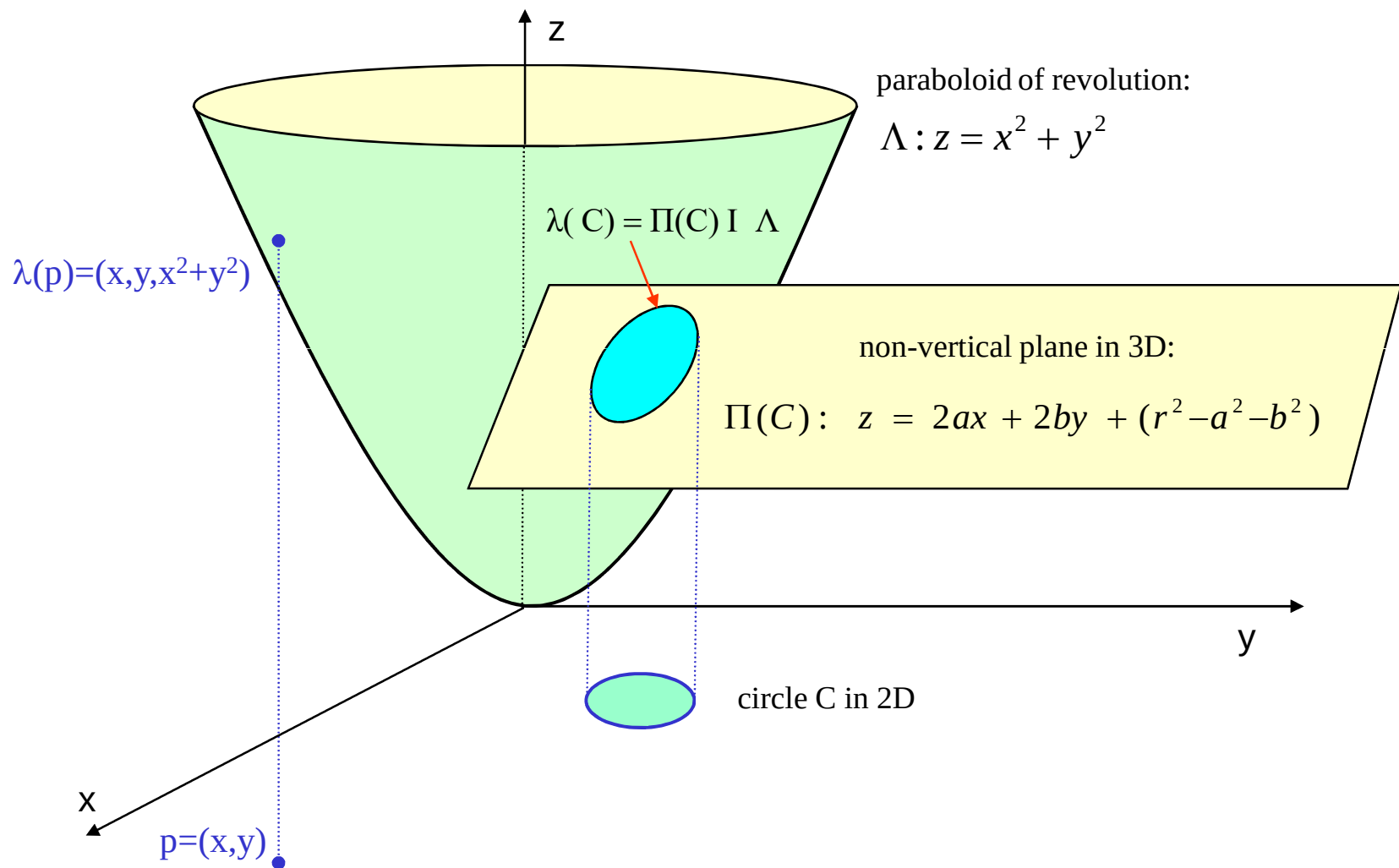


□ Circle C with center (a,b) and radius r : $(x-a)^2 + (y-b)^2 = r^2$

$$\Leftrightarrow x^2 + y^2 = 2ax + 2by + (r^2 - a^2 - b^2)$$

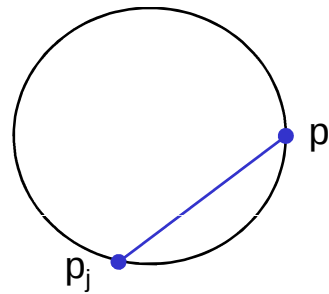
$$\left\{ \begin{array}{ll} z = x^2 + y^2 & : \Lambda \\ z = 2ax + 2by + (r^2 - a^2 - b^2) & : \Pi(C) \end{array} \right.$$

Lifting from 2D to 3D



Relaxing the empty circle property

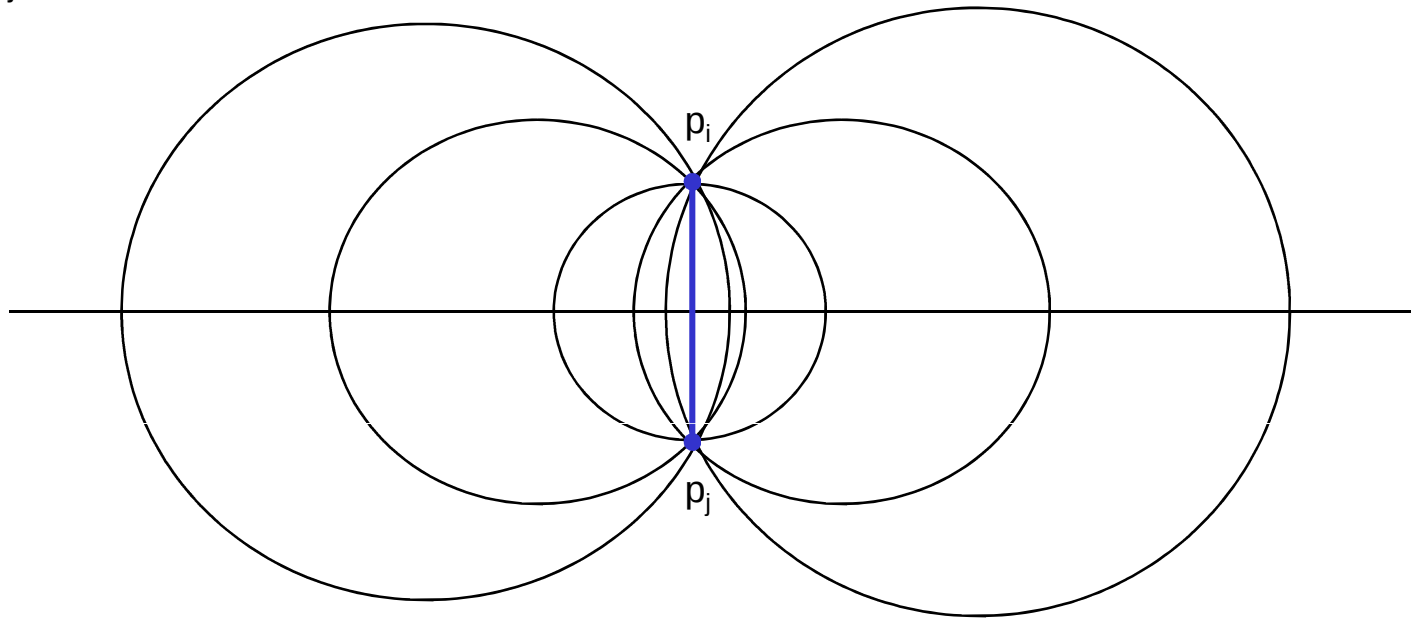
- If $p_j = \text{NN}(p_i)$, then the circle with diameter (p_i, p_j) is an empty circle.
- A Delaunay edge is any pair (p_i, p_j) that is a chord of some empty circle.
(Note that (p_i, p_j) is any chord, not necessarily a diagonal, of that circle.)



- The Delaunay graph is a super-graph of ANNG, and a sub-graph of the complete graph on P .

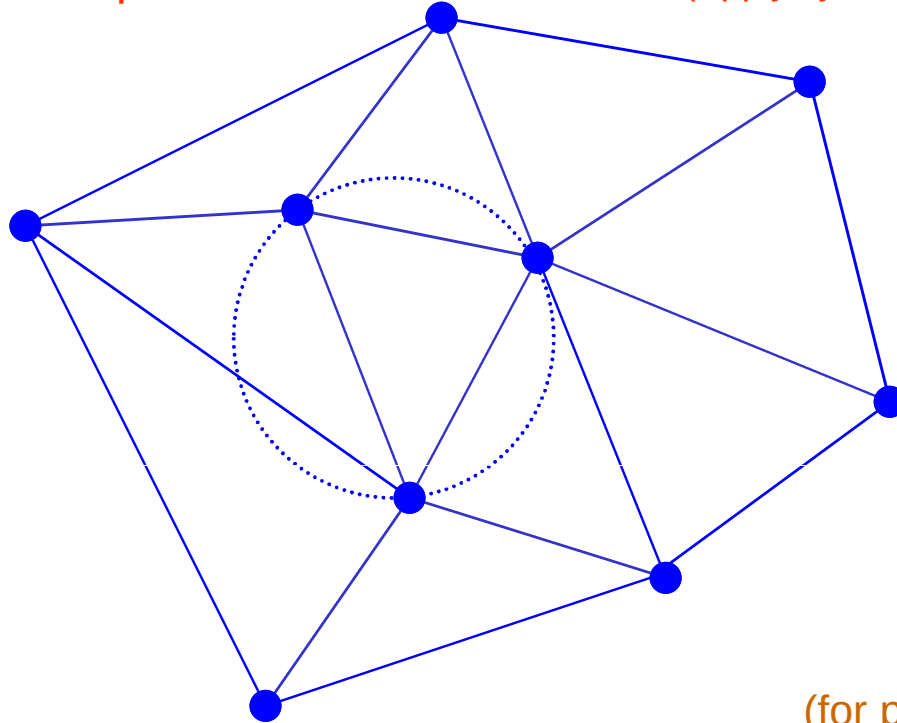
Delaunay Graph . . .

□ (p_i, p_j) as chord of many circles.



Delaunay Graph is a Triangulation of P

Assumption: No 4 points of P are co-circular. (Apply symbolic perturbation.)



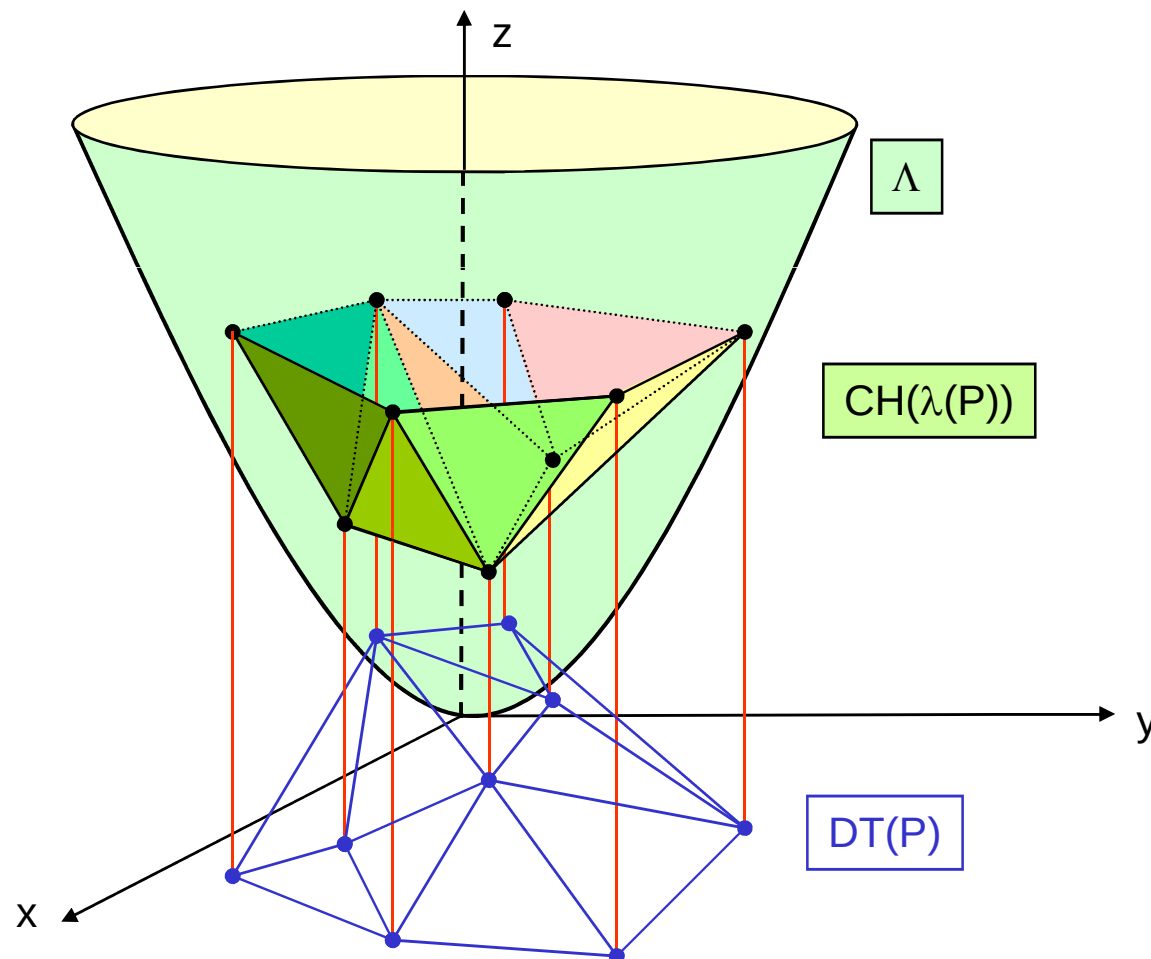
CLAIM:

(for proof see next slides)

- ❑ Delaunay Triangulation $DT(P)$ partitions $CH(P)$ into triangles with vertex set P.
- ❑ Delaunay triangles are precisely those whose circumscribing circles are empty.
- ❑ $DT(P)$ is a planar graph with n vertices, hence, it has $\leq 3n$ edges.
- ❑ $DT(P)$ is a super-graph of ANNG.

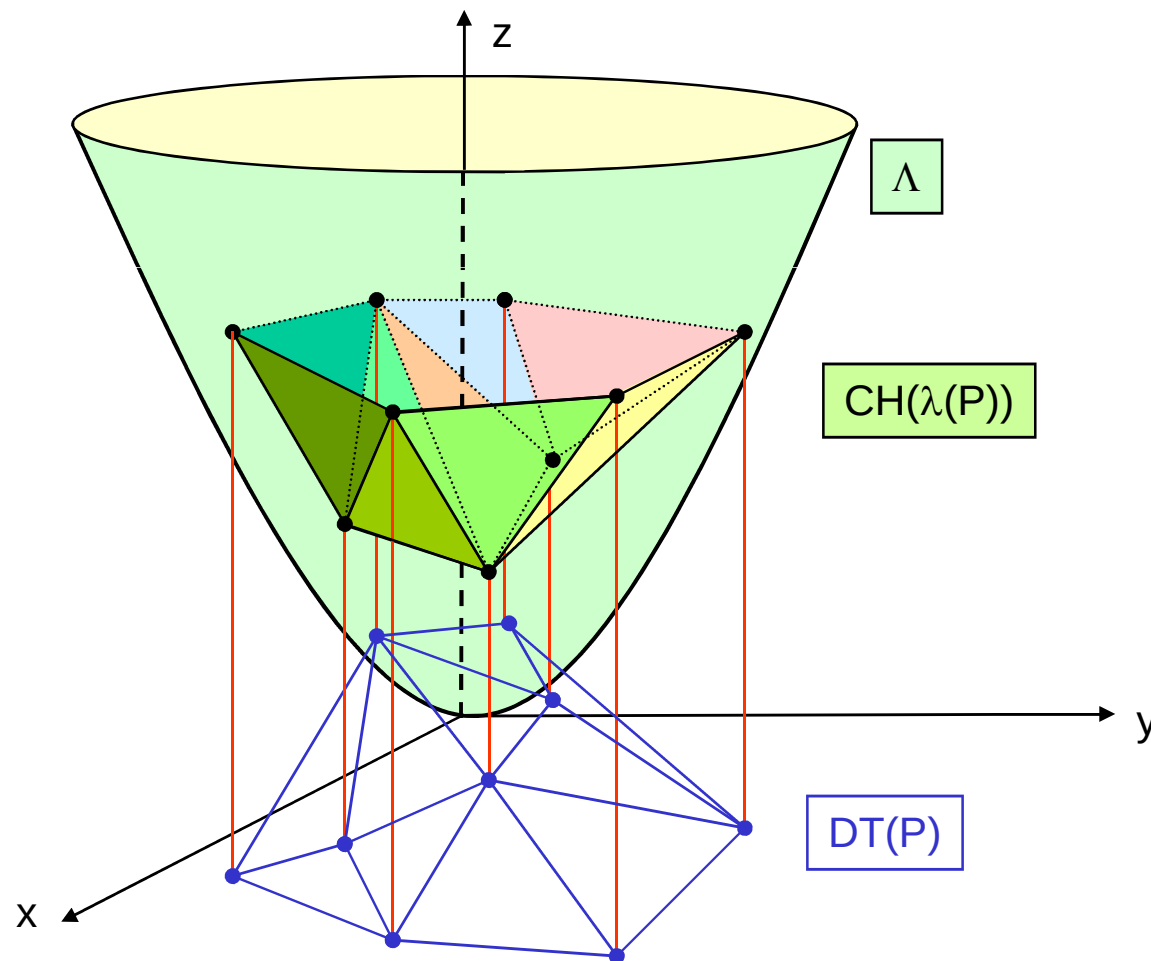
DT(P) as projection of $CH(\lambda(P))$

- ❑ Vertically lift each point $p=(x,y)$ from 2D to point $\lambda(p) = (x,y, x^2+y^2)$ on Λ in 3D.
- ❑ $\lambda(P) = \{ \lambda(p) \mid p \in P \}$.
- ❑ $CH(\lambda(P))$ is convex hull of $\lambda(P)$ in 3D. (Only “lower” faces shown below.)
- ❑ $DT(P)$ = Delaunay Triangulation of P in 2D = projection of lower hull $CH(\lambda(P))$.



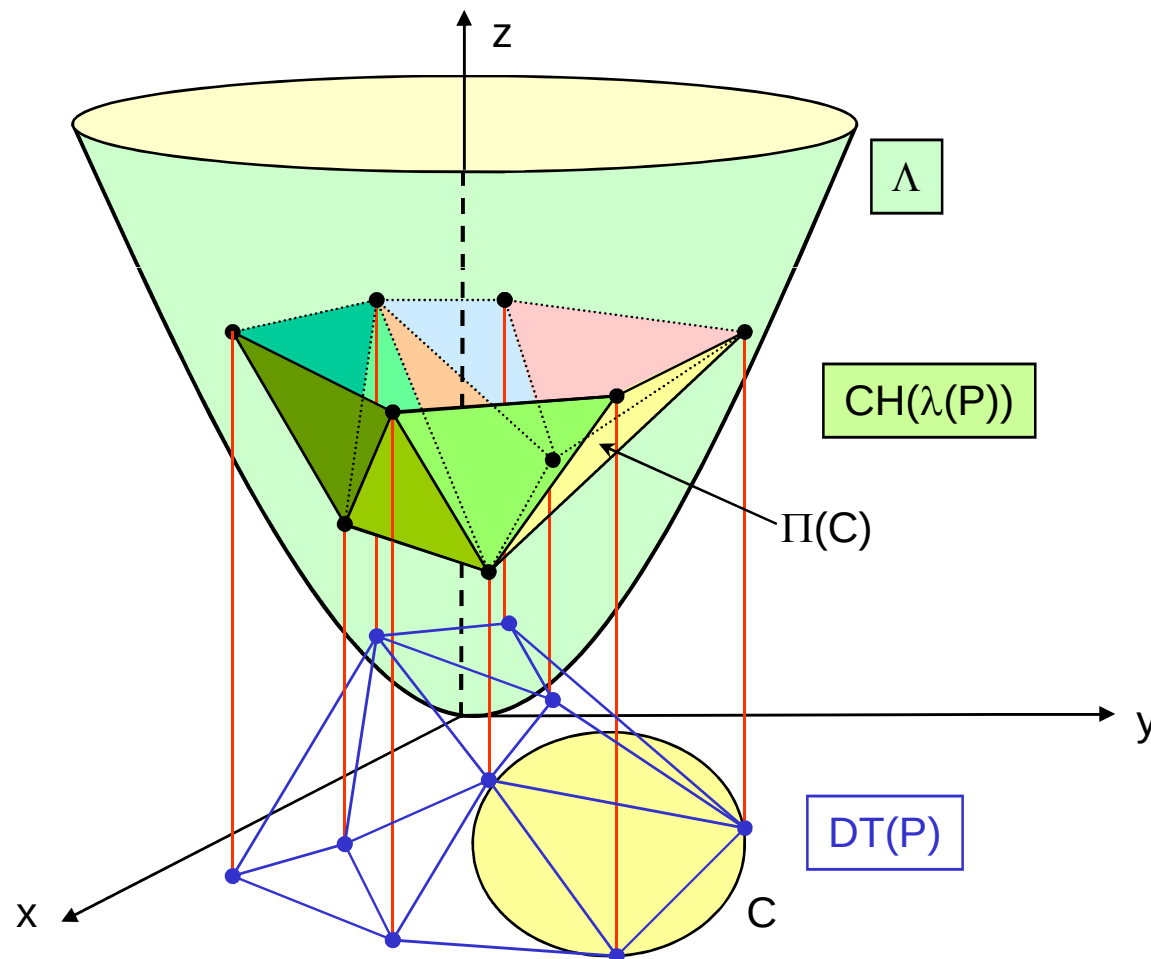
DT(P) as projection of $\text{CH}(\lambda(P))$

- Λ is a convex surface. So, all points $\lambda(p)$ of $\lambda(P)$ are extreme (vertices of $\text{CH}(\lambda(P))$), since the tangent plane to Λ at $\lambda(p)$ is a supporting plane of $\lambda(P)$.



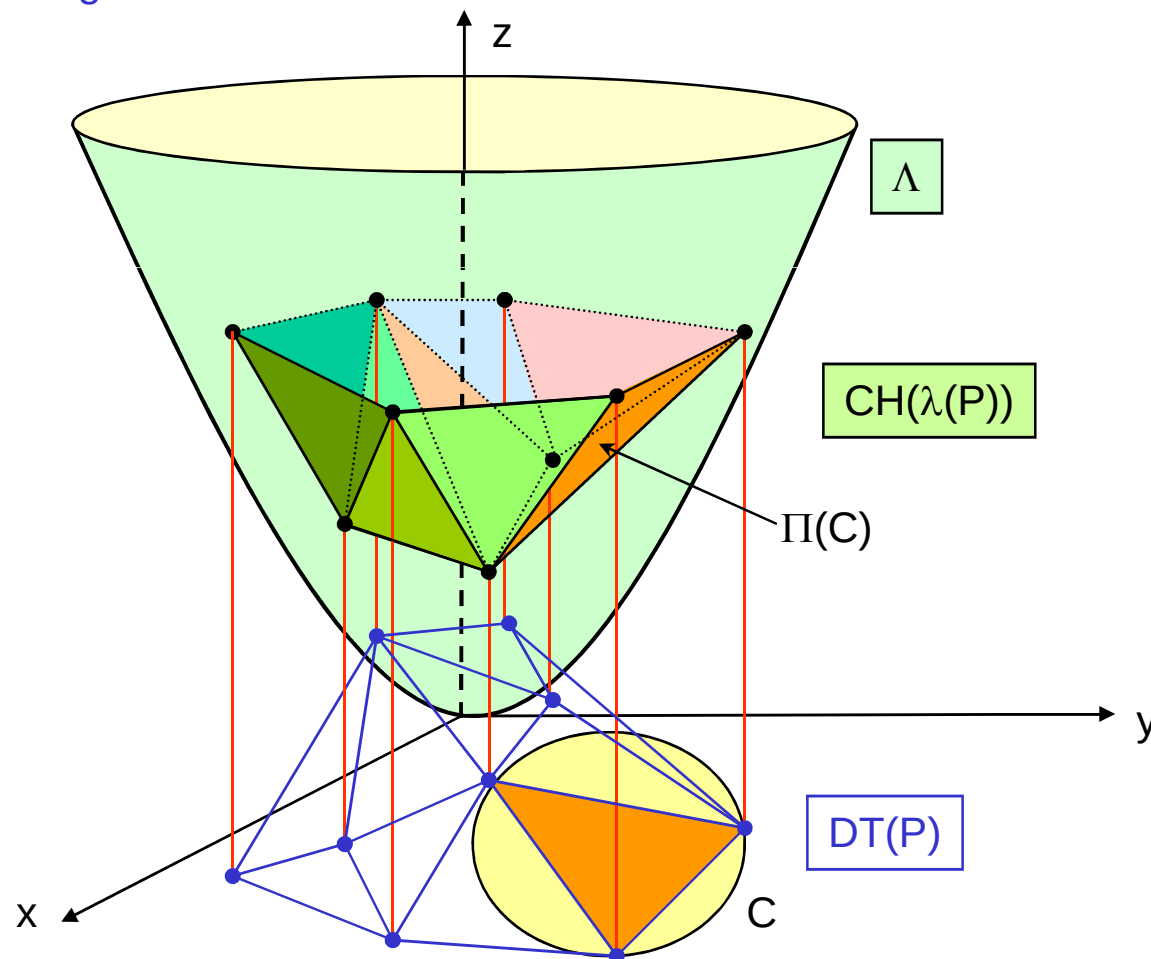
DT(P) as projection of $CH(\lambda(P))$

- Each Delaunay edge (p_i, p_j) is chord of some empty circle C .
- $p \in P$ is outside C if and only if $\lambda(p)$ (on convex Λ) is above plane $\Pi(C)$.
- C is empty, so $\lambda(P - \{p_i, p_j\})$ is above $\Pi(C)$, and $\lambda(p_i)$ & $\lambda(p_j)$ are on $\Pi(C)$.
- So, $\Pi(C)$ is a supporting plane of edge $(\lambda(p_i), \lambda(p_j))$ of the 3D $CH(\lambda(P))$ from below.



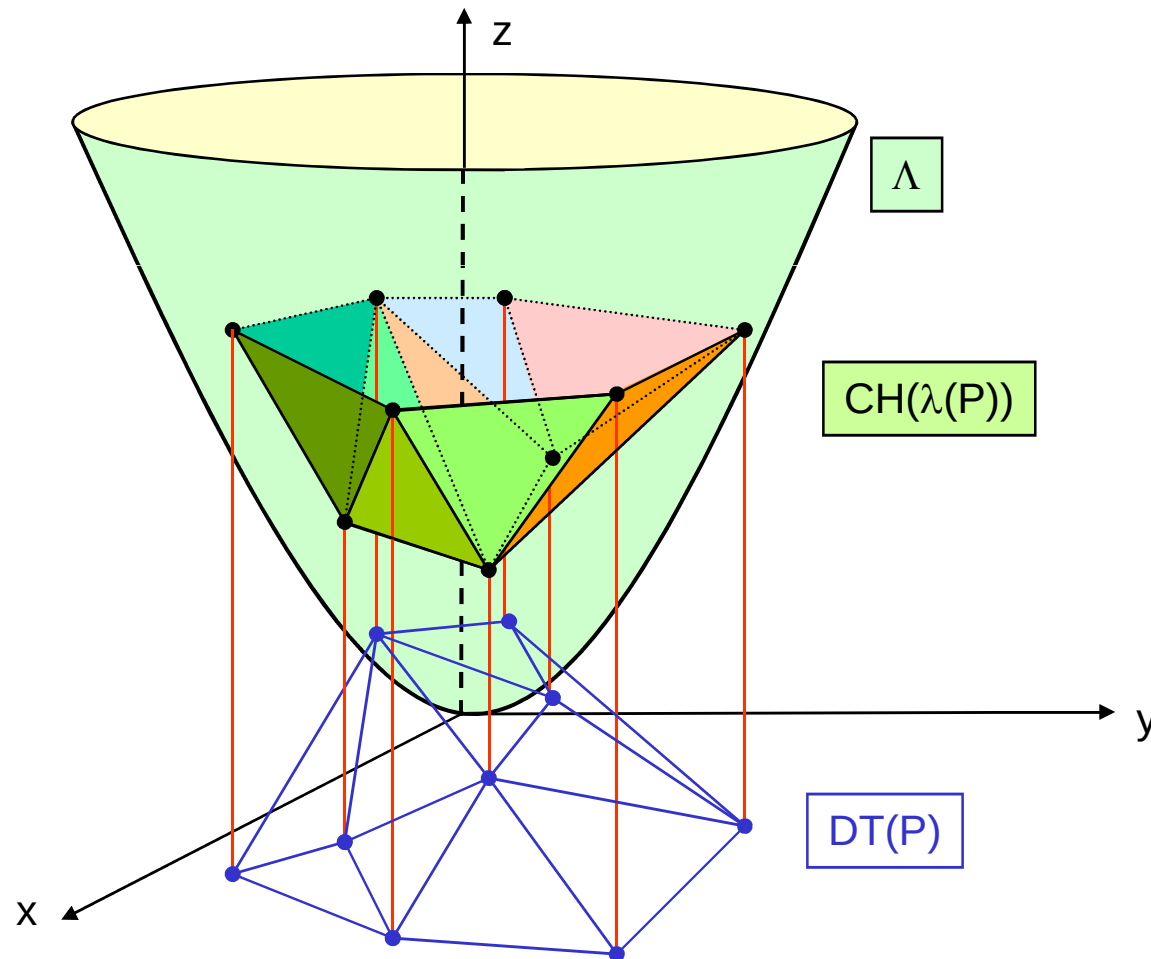
DT(P) as projection of $\text{CH}(\lambda(P))$

- ❑ So, $\text{DT}(P)$ is the projection of the “lower” convex hull edges of $\text{CH}(\lambda(P))$.
- ❑ $\text{CH}(P)$ is the shadow of $\text{CH}(\lambda(P))$ on the xy -plane.
- ❑ No 4 points of P are co-circular. So, no 4 points of $\lambda(P)$ are co-planar.
- ❑ So, “lower” facets of $\text{CH}(\lambda(P))$ are triangles & project down to the xy -plane as the Delaunay triangles.



DT(P) as projection of $\text{CH}(\lambda(P))$

- ❑ So, $\text{DT}(P)$ is a triangulation of P whose triangles have the empty-circle property.
- ❑ 3D CH of n points can be computed in $O(n \log n)$ time (e.g., by divide-&-conquer).
- ❑ So, $\text{DT}(P)$ can be computed in $O(n \log n)$ time.
- ❑ So, $\text{ANNG}(P)$ can be computed in $O(n \log n)$ time.



Exercises

1. Show that the equation of the unique circle that passes through three given points $p_i = (x_i, y_i)$, $i=1..3$, is

$$\det \begin{bmatrix} x_1 & y_1 & x_1^2 + y_1^2 & 1 \\ x_2 & y_2 & x_2^2 + y_2^2 & 1 \\ x_3 & y_3 & x_3^2 + y_3^2 & 1 \\ x & y & x^2 + y^2 & 1 \end{bmatrix} = 0.$$

2. Let P be a set of n points in the Euclidean plane. $\text{EMST}(P)$, the Euclidean Minimum Spanning Tree of P , is the Minimum Spanning Tree of the complete graph on the vertex set P with Euclidean edge lengths. Considered as a set of undirected edges, show the following set inclusions: $\text{ANNG}(P) \subseteq \text{EMST}(P) \subseteq \text{DT}(P)$.
3. We showed the Closest Pair and All Nearest Neighbors problems can be solved in $O(n \log n)$ time in the L_2 metric. How would you solve these problems in the L_∞ metric? How about the L_1 metric?
4. Given a set P of n points in the plane, develop efficient algorithms for the following:
 - (a) find the largest empty circle C with center inside $\text{CH}(P)$.
 - (b) find the largest empty axis-parallel square S with center inside $\text{CH}(P)$.
 - (c) find the smallest circle that contains all n points of P .

5. **Starbucks vs Tim Hortons:** There are n Tim Hortons sites in Hyperville. The Starbucks company has hired you as consultant to locate the opening site of their first Starbucks coffee shop in that city. The requirement is that the new site is restricted to be within the city limits but as far away from its nearest competitor as possible.

Formulated as a computational geometry problem, you are given a set P of n points in the plane (the Tim Hortons sites). Your problem is to find a point q (the Starbucks site) anywhere on the boundary or in the interior of convex hull of P , so that it maximizes $\min_{p \in P} \text{dist}(p, q)$, where $\text{dist}(p, q)$ is assumed to be the Euclidean distance between points p and q .

Design and analyze an efficient algorithm for this problem.

6. **Closest Red Blue Pair:** We are given a set of n points in the plane. Each point is coloured either **red** or **blue**. Design, analyze and prove correct an efficient algorithm to determine the closest **red-blue** pair (i.e., the closest pair among the input points such that one is **red** the other **blue**).

END